PRML 作业 1 报告

23231044@buaa. edu. cn
报告人：武俊成

摘要

此次作业分别使用**最小二乘法**，**梯度下降法(GD)**和**牛顿法**来对数据进行**线性拟合**，观察其训练误差与测试误差。除此之外还使用了**五阶傅里叶级数**（Fourier series）作为基底通过 **GD** 进行对应系数拟合。最终得出拟合函数图、相应均方差以及损失值等数据并进行了相应的分析。

引言

在之前的课堂上我们学习了最小二乘法、梯度下降法以及牛顿法算法的思想以及相应线性拟合的数学原理，还学习了损失函数（Loss Function）的定义以及作用，还了解到机器学习是如何逐渐优化相应参数，从而使 Loss 值不断下降。除此之外还了解到了过拟合、距离定义等概念。本次作业将通过 python 展示上述几种模型的拟合过程以及相应的拟合效果。

研究方法

本次作业使用 python 语言进行编程，四个算法均使用了 os 包进行路径的读取，通过其访问同文件下的 DATA. xlsx 以及 TEST. xlsx 数据集和测试集，并通过 pandas 包进行表格数据的输入，最后使用 matplotlib 包进行相应图像的绘制。

方法 1 ：最小二乘法

最小二乘通过最小化模型预测值与真实观测值之间的误差平方和（Sum of Squared Errors, SSE），来寻找数据的最佳线性函数匹配。最小二乘的误差函数选取的是方差误差，示意为：

$$L(\theta) = \sum_{i=1}^{N}(y_i - \hat{y}_i)^2 = (y - X\theta)^T(y - X\theta) \cdots\cdots\cdots\cdots\cdots\cdots \quad (1)$$

之后再对误差函数 $\boldsymbol{L}$（$\boldsymbol{\theta}$）求偏导，并令其等于 0，即：

$$\frac{\partial L(\theta)}{\partial \theta} = -2X^T(y - X\theta) = 0 \cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots \quad (2)$$

最后再对将其展开并移项化简后得到正规方程：

$$\theta = (X^T X)^{-1} X^T y \cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots \quad (3)$$

代码实现仅需进行一个矩阵乘法以及矩阵的逆的运算即可，仅需注意矩阵运算前可以使用 np. linalg. det（A） == 0 来判断行列式是否可逆，若可逆则正常进行运算即可。

方法 2：梯度下降法

梯度下降法是一种基于一阶导数的迭代优化算法。与最小二乘不同的是，梯度下降法其算法通过不断计算损失函数关于参数的梯度，并沿着梯度下降的方向一步步微调参数，最终逼近全局最优解或局部最优解。

算法采用均方误差（MSE）作为损失函数：

$$L(w, b) = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2 \cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots \quad (4)$$

在每次迭代中，算法将计算当前预测值与真实值之间的误差向量

$$error = \hat{y} - y \cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots \quad (5)$$

之后对权重 $\boldsymbol{w}$ 和截距 $\boldsymbol{b}$ 分别求偏导数，得到梯度公式：

$$dw = \frac{2}{N} X^T \cdot error \cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots (6)$$

$$db = \frac{2}{N} \sum error \cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots (7)$$

最终分别对 $\boldsymbol{w}$ 以及 $\boldsymbol{b}$ 进行调整便完成一次循环。在模型初始化时，设置了学习率 lr=0.01 和总迭代次数 epochs=5000。在 fit 函数中，权重被初始化为全零向量，截距初始化为 0。

同时，为了监控模型的收敛过程，代码在每次迭代末尾计算当前的均方误差，并将其追加到列表中。同时，程序还设置了一系列特定的检查点（第 10, 50, 100, 200, 500, 600, 700, 800, 900, 1000, 2000, 3000, 4000, 5000 次 epoch），并最终在终端打印输出特定阶段的 loss 值。

方法三：牛顿法

牛顿法是一种基于二阶导数（Hessian 矩阵）的高效迭代优化算法。与仅依靠一阶导数（坡度）决定方向的梯度下降法不同，牛顿法通过计算二阶导数引入了对曲面弧度（曲率）的评估，从而能够更加精准地预测极值点的位置，其收敛速度通常远快于梯度下降法。

牛顿法算法构建中依然采用均方误差作为损失函数。为了寻找使损失函数最小的参数 θ，需要将目标函数在其当前迭代点 θ 处进行二阶泰勒展开：

$$L(\theta) \approx L(\theta_t) + \nabla L(\theta_t)^T (\theta - \theta_t) + \frac{1}{2} (\theta - \theta_t)^T H (\theta - \theta_t) \cdots \quad (8)$$

通过对该近似函数求导并令其为 0，其参数更新公式为：

$$\theta_{t+1} = \theta_t - H^{-1} \nabla L(\theta_t) \cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots \quad (9)$$

其二阶导数（海森矩阵）为一个常数矩阵：

$$H = \frac{2}{N} X^T X \cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots (10)$$

在代码的具体实现中，模型的训练架构与梯度下降法类似，依然保留了迭代机制以适应更通用的优化框架。在每次 epoch 中，代码首先计算当前的预测

误差，进而求得梯度向量与海森矩,然后通过将梯度信息与海森矩阵相乘来更新权重向量。同时，为了监控模型的收敛过程，代码在每次迭代末尾计算当前的均方误差，并将其追加到列表中。与上相同，算法依旧设置了一系列特定的检查点（第 10，50，100，200，500，600，700，800，900, 1000, 2000，3000，4000，5000 次 epoch），并最终在终端打印输出特定阶段的 loss 值。

方法四：傅里叶基底

通过观察数据散点图（详细见图（1））发现数据具有一定三角函数性质，于是可以使用五级傅里叶级数作为基底，通过梯度下降法进行相应系数的确定。代码实现方面仅需改动梯度下降算法中的基底即可。

实验结果与分析

实验结果如下：

Table 1 实验结果表

Method	Train MSE	Test MSE	Intercept	Weight
OLS	0. 6134	0. 6134	−0. 6487	0. 1089
GD	0. 6134	0. 6134	−0. 6487	0. 1089
Newton	0. 6134	0. 6134	−0. 6487	0. 1089
Fourier	0. 3652	0. 3652	−0. 1062	

对于 Fourier，其 weight 按照 ***sin*（*wx*）、*cos*（*wx*）**顺序为：（-0.33 0.3365，-0.4475，-0.0446，-0.1772，0.0622，0.4756，-0.0204，0.0768，-0.1102）

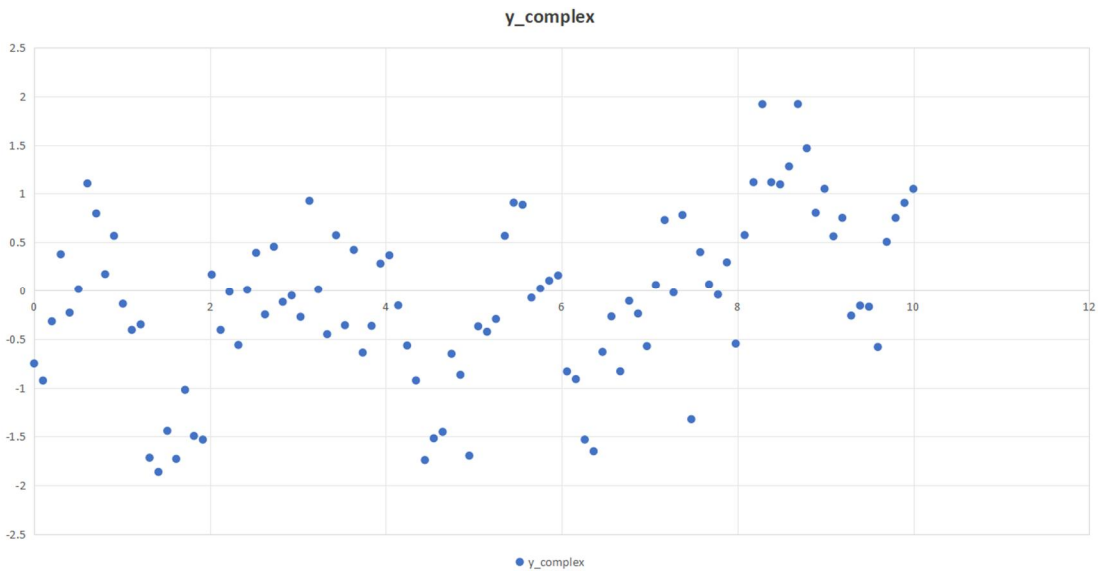

Figure 1 训练数据集散点图

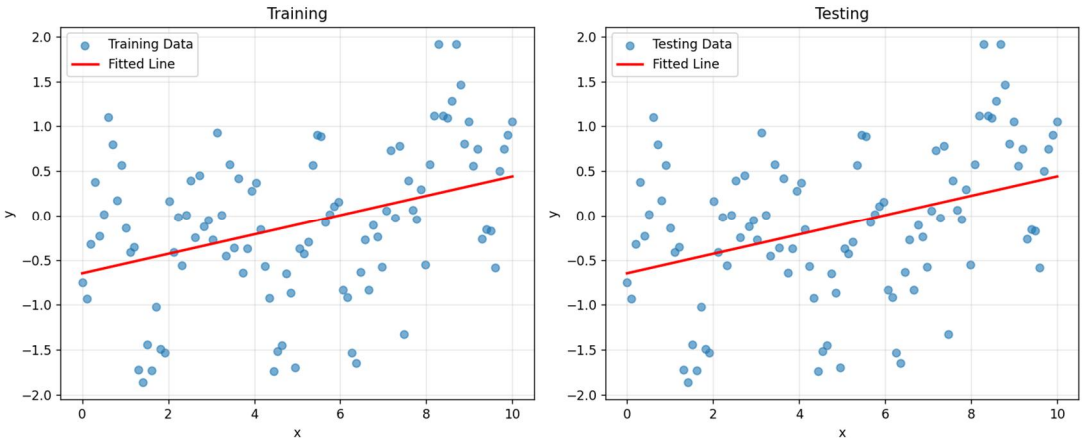

Figure 2 OLS 算法拟合图

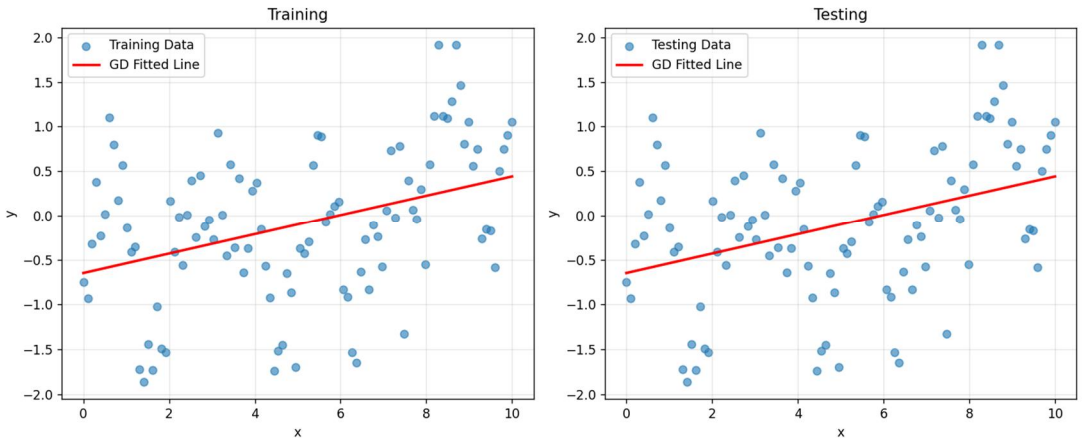

Figure 3 GD 算法拟合图

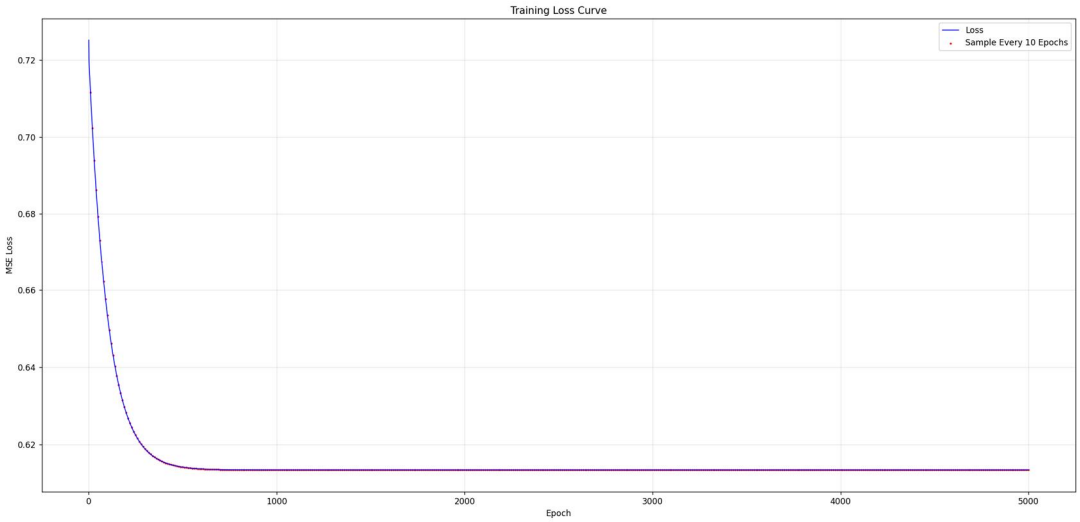

Figure 4 GD 算法 LOSS 变化图

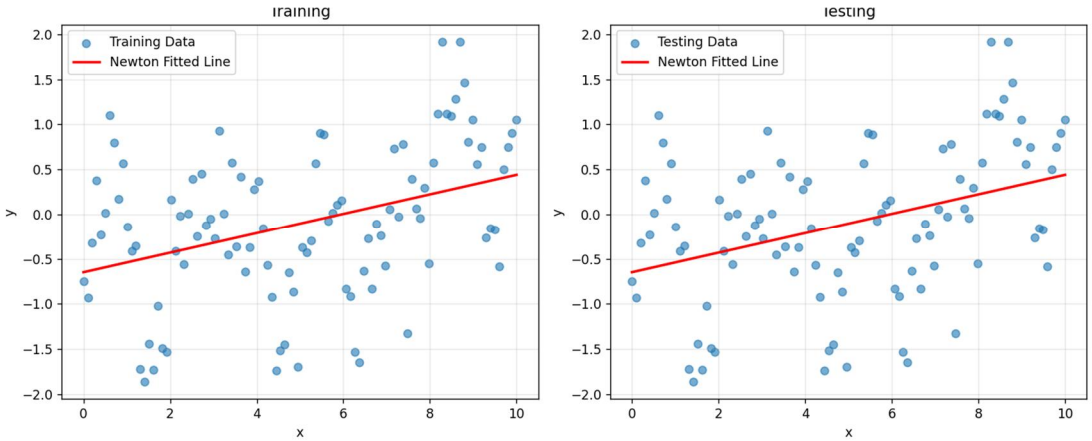

Figure 5 牛顿法拟合图

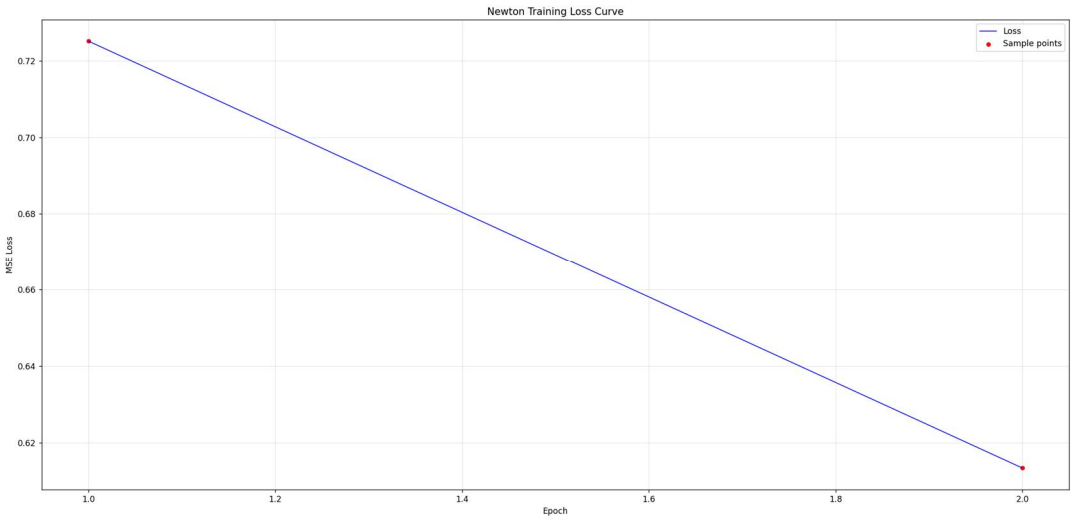

Figure 6 牛顿法 LOSS 变化图

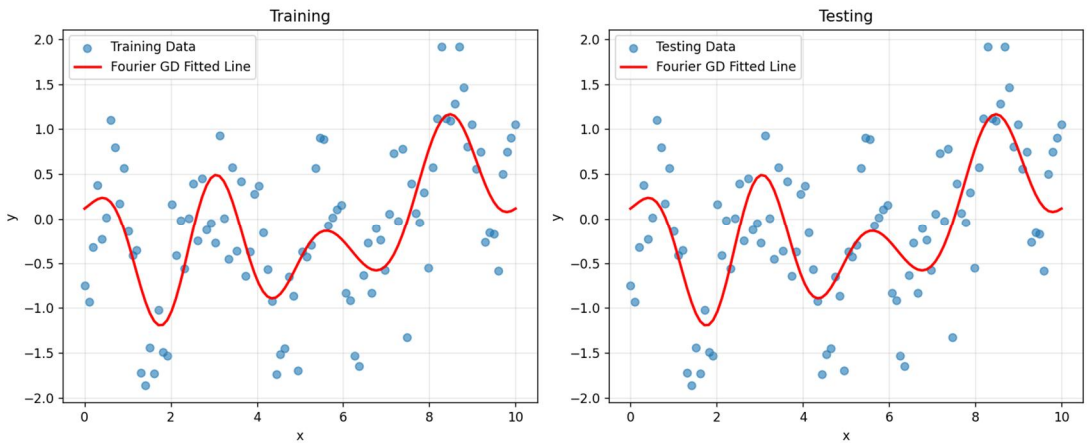

Figure 7 Fourier（GD）拟合图

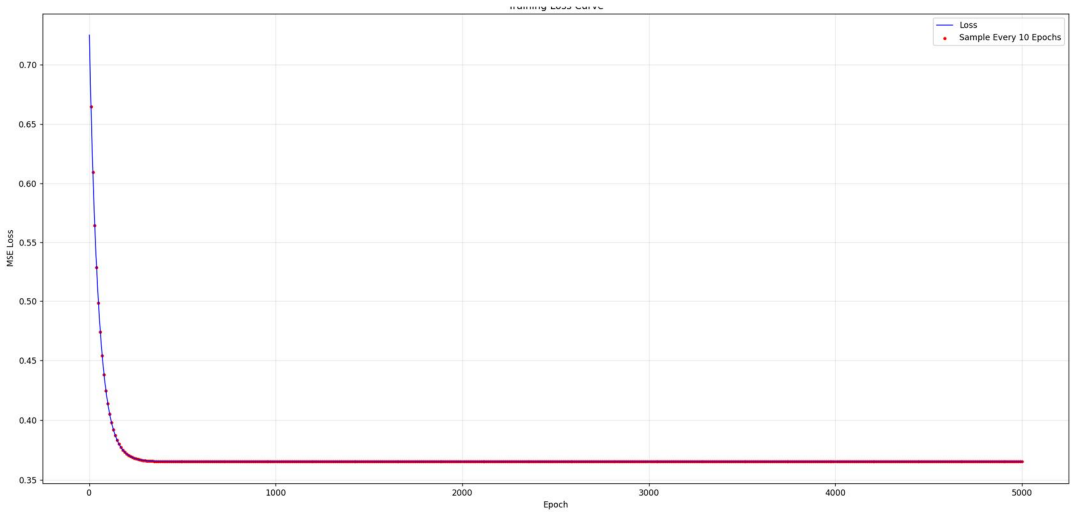

Figure 8 Fourier（GD）LOSS 变化图

结论

通过上述实验结果分析，可以知道 OLS、GD 以及 Newton 三者线性拟合最终结果完全相同，均为-0. 6487 的截距与 0. 1089 的斜率。同时三者拟合均方差都较差（0. 6134），说明提供数据并非线性关系。三者差距体现于计算量以及 Loss 收敛速度上，最小二乘法（OLS）通过正规方程直接求得解析解，无需迭代，但在处理海量高维特征时，矩阵求逆的计算成本极高；梯度下降法（GD）作为一阶优化算法，单次迭代计算量小，适合大规模数据，但需要设定合适的学习率并经过大量迭代才能缓慢收敛；而牛顿法利用了二阶导数（海森矩阵）信息，对于线性回归这种具有二次凸损失函数曲面的模

型，能够快速达到全局最优解，但其代价是每次计算海森矩阵的逆同样面临较高的算力瓶颈。

除此之外，使用傅里叶级数作为基底，使用（GD）进行计算所得到的均方差比线性拟合效果有很好的提高（0. 3652），而且根据 Loss 变化图可知其收敛速度也显著优于线性拟合的 GD 算法。